

Pinia in Vue.js – বাংলায় সম্পূর্ণ ডকুমেন্টেশন

Pinia কী?

Pinia হলো Vue 3-এর **official state management library**। সহজ ভাষায় বললে:

👉 Pinia ব্যবহার করা হয় একাধিক component-এর মধ্যে data (state) শেয়ার করার জন্য।

যেমন: - User login তথ্য - Game score - Shopping cart - Theme (dark/light)

যেসব data এক জায়গায় রেখে পুরো app-এ ব্যবহার করতে হয়, সেগুলোর জন্য Pinia খুব দরকারি।

কেন Pinia দরকার? (Problem বুঝি আগে)

ধরি, আপনার কাছে একটি Vue App আছে:

```
App.vue
├─ Navbar.vue
├─ Game.vue
└─ Footer.vue
```

Requirement (Real-life Example)

- Navbar.vue → current score দেখাবে
- Game.vue → score বাড়াবে/কমাবে
- Footer.vue → max score দেখাবে

👉 এখন প্রশ্ন: এই score কোথায় রাখবেন?

Pinia ছাড়া কী সমস্যা হয়?

Props Drilling Problem

```
App.vue
└─ props
  Game.vue
```

↓ props
Child.vue

- অনেক level নিচে data পাঠাতে হয়
- Code messy হয়
- Maintain করা কঠিন

Emit Hell

- Child → Parent → Grand Parent
- Event tracking কঠিন

provide / inject সীমাবদ্ধ

- Debug করা কঠিন
- Global logic manage করা যায় না

Pinia কীভাবে সমস্যার সমাধান করে?

Pinia ব্যবহার করলে:

 একটি **Central Store** থাকে

Components ↔ Pinia Store

- সব component একই store থেকে data নেয়
- যেকোনো component data update করতে পারে
- Reactive & fast

Real World Example: Game Score System

Scenario

একটি Game App: - Player score থাকবে - Attack করলে score বাড়বে - Defense করলে score কমবে - Random action থাকবে

 এই score সব component থেকে access করা দরকার

এখানেই Pinia দরকার।

Pinia কিভাবে কাজ করে? (Internally)

Pinia মূলত ৩টা জিনিস নিয়ে কাজ করে:

State

```
state: () => ({
  score: 0
})
```

👉 Data রাখার জায়গা (Reactive)

Getters

```
getters: {
  isGameOver: (state) => state.score <= 0
}
```

👉 Computed value

Actions

```
actions: {
  increment() {
    this.score++
  }
}
```

👉 Logic / Methods

Step-by-Step Pinia Implementation

Pinia Install

```
npm install pinia
```

main.js এ Pinia Register

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

createApp(App)
  .use(createPinia())
  .mount('#app')
```

Store Create করা (stores/gameStore.js)

```
import { defineStore } from 'pinia'

export const useGameStore = defineStore('game', {
  state: () => ({
    score: 0,
    maxAttack: 10,
    maxDefense: 5,
    maxHealth: 100
  }),

  getters: {
    healthStatus: (state) =>
      state.score >= state.maxHealth ? 'FULL' : 'NOT FULL'
  },

  actions: {
    incrementScore() {
      this.score += Math.ceil(Math.random() * this.maxAttack)
    },
    decrementScore() {
      this.score -= Math.ceil(Math.random() * this.maxDefense)
    },
    randomScore() {
      Math.random() > 0.5
        ? this.incrementScore()
        : this.decrementScore()
    }
  }
})
```

Component এ Store ব্যবহার

```
<script setup>
import { useGameStore } from '@stores/gameStore'

const gameStore = useGameStore()
</script>

<template>
  <h2>Score: {{ gameStore.score }}</h2>
  <button @click="gameStore.incrementScore">Attack</button>
  <button @click="gameStore.decrementScore">Defense</button>
  <button @click="gameStore.randomScore">Random</button>
</template>
```

Data Flow (সহজ ভাষায়)

Component → Action → State Update → UI Auto Update

- Action call হয়
- State change হয়
- Vue reactivity UI আপডেট করে

👉 Manual DOM update লাগে না

Pinia vs provide/inject (Bangla)

বিষয়	Pinia	provide/inject
Global state	👉	👉
Debugging	👉 Devtools	👉
Structure	Clean	Messy
Best for	Large apps	Small scope

কখন Pinia ব্যবহার করবেন?

👉 Use Pinia when: - Multiple component একই data ব্যবহার করে - App বড় হচ্ছে - Authentication / Cart / Game logic আছে

👉 Use Pinia করবেন না যখন: - শুধু একটি component-এর local state

📦📦 Conclusion (সংক্ষেপে)

👉 Pinia হলো Vue 3-এর **brain** 🧠

- Central data management
- Clean architecture
- Easy debugging
- Production ready

Professional Vue Developer হতে Pinia জানা MUST 👉

আপনি চাইলে আমি পরের ধাপে দেখাতে পারি: - 😊 Auth system with Pinia - 📦 localStorage persistence - 🙌 API call + Pinia - 🐱 Real project structure

বললেই হবে 😭